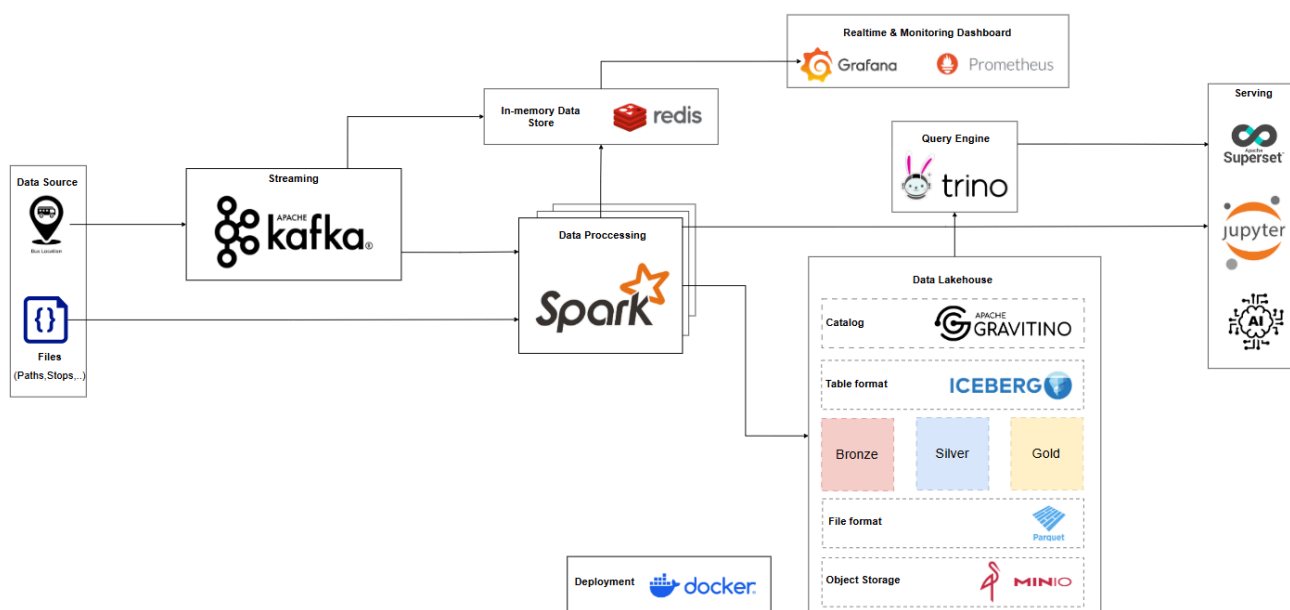


Tầng Phục vụ Redis (Redis Serving Layer) - Truy cập Dữ liệu Thời gian thực

Tài liệu này chi tiết hóa việc triển khai và ứng dụng Redis trong kiến trúc Nền tảng Dữ liệu (Data Platform), đóng vai trò là **Tầng phục vụ (Serving Layer)** cho các ứng dụng thời gian thực.



1. Vai trò của Redis trong kiến trúc

Trong khi Iceberg (Data Lakehouse) phục vụ các truy vấn phân tích sâu (Batch/Analytical), Redis được sử dụng để:

- **Độ trễ cực thấp (Low Latency):** Cung cấp dữ liệu với độ trễ < 10ms.
- **Giám sát thời gian thực (Real-time Monitoring):** Theo dõi vị trí và trạng thái xe buýt tức thời.
- **Tổng hợp KPI nhanh:** Hiển thị các chỉ số tổng hợp (số xe hoạt động, tốc độ trung bình) theo từng tuyến đường.

2. Cấu trúc dữ liệu và Triển khai

Hệ thống sử dụng linh hoạt các cấu trúc dữ liệu của Redis để tối ưu hiệu năng:

2.1 Redis Stream (`buswaypoint_stream`)

- **Mục đích:** Lưu trữ luồng sự kiện (event stream) thô từ Kafka.
- **Chi tiết:**
 - Sử dụng tính năng `maxlen` để giới hạn bộ nhớ (khoảng 3.6 triệu bản ghi, tương đương 1 giờ dữ liệu với tốc độ 1000 msg/s).
 - Phục vụ các ứng dụng cần tiêu thụ lại dữ liệu thời gian thực hoặc hiển thị lịch sử ngắn hạn trên bản đồ.

2.2 Redis Hashes (`buswaypoint_latest:{vehicle_id}`)

- **Mục đích:** Lưu trữ trạng thái mới nhất của từng phương tiện (Latest Known State).
- **Chi tiết:**
 - Truy cập cực nhanh với độ phức tạp $O(1)$ thông qua `vehicle_id`.
 - Chứa các trường: `speed`, `latitude`, `longitude`, `ignition`, `aircon`, `route_no`, `updated_at`.
 - **TTL:** Tự động hết hạn sau 2 giờ không có hoạt động để tiết kiệm bộ nhớ.

2.3 Redis Hashes (`bus_metrics:route:{route_no}`)

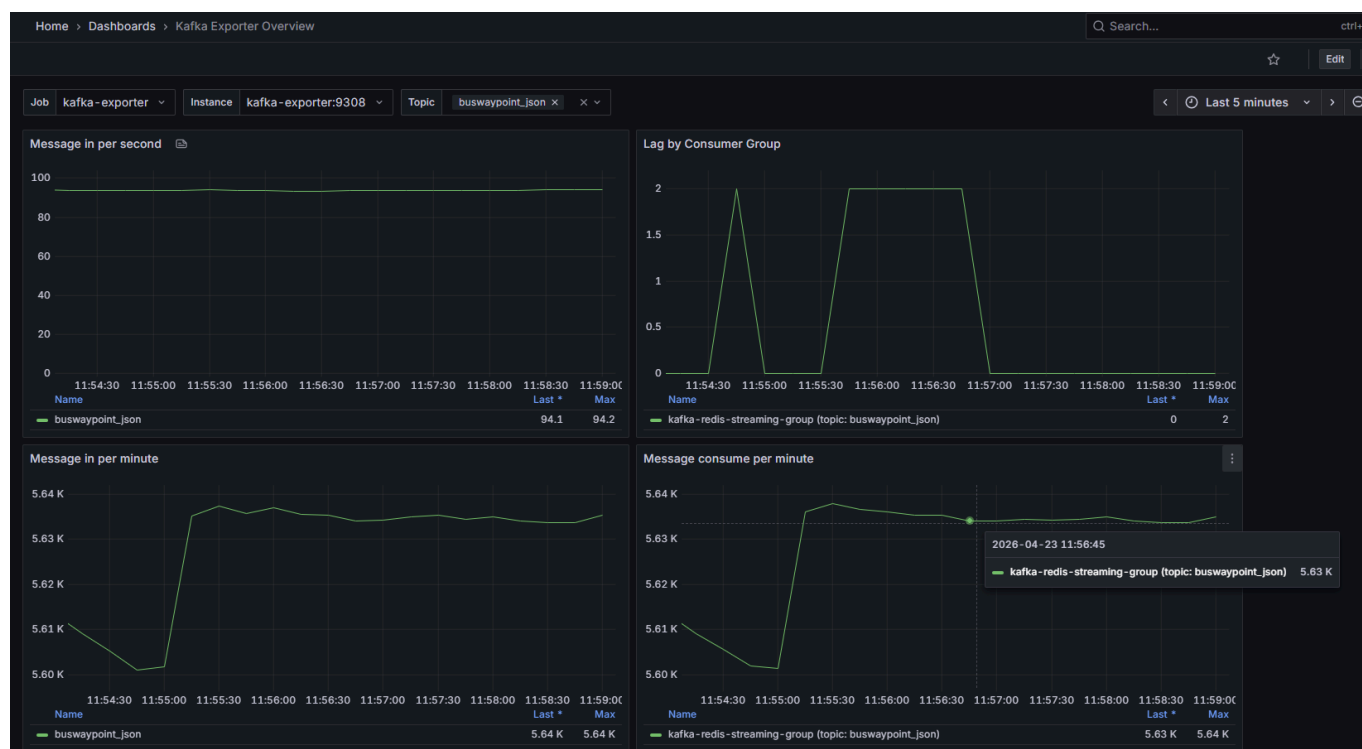
- **Mục đích:** Lưu trữ các chỉ số KPI đã được tổng hợp sẵn cho từng tuyến xe.
- **Cơ chế:** Được cập nhật định kỳ sau mỗi batch xử lý bằng cách quét qua trạng thái mới nhất của tất cả các xe (`buswaypoint_latest:*`).
- **Các chỉ số (Fields):**
 - `active_vehicle_count`: Số lượng xe đang hoạt động (có tín hiệu `ignition=True`).
 - `idle_vehicle_count`: Số lượng xe đang dừng (vận tốc `speed=0`).
 - `sos_vehicle_count`: Số lượng xe đang ở trạng thái khẩn cấp (SOS).
 - `aircon_on_count`: Số lượng xe đang bật điều hòa.
 - `avg_speed`: Tốc độ trung bình trên toàn tuyến.
 - `updated_at`: Thời điểm cập nhật cuối cùng.

2.4 Redis Sets (`vehicles_seen`, `routes_active`)

- **Mục đích:** Quản lý danh sách định danh (ID) các phương tiện và tuyến đường đang hiện diện.
- **Ứng dụng:** Làm nguồn dữ liệu cho các bộ lọc (Dropdown) trên Dashboard Grafana.

3. Quy trình xử lý (Pipeline): Kafka to Redis (Python)

Thực hiện bởi `kafka_to_redis.py`, tập trung vào hiệu năng cao:



1. **Làm phẳng dữ liệu (Flattening):** Chuyển đổi JSON phức tạp sang dạng key-value phẳng cho Redis Hash.

2. **Làm giàu dữ liệu (Enrichment)**: Ánh xạ xe vào tuyến đường ngay khi nhận dữ liệu từ Kafka.
3. **Xử lý theo lô (Pipelines)**: Gom nhóm các lệnh ghi để giảm thiểu Round-trip time (RTT).
4. **Tính toán Snapshot**: Tính toán KPI tổng hợp từ các bản ghi trạng thái mới nhất thay vì quét toàn bộ lịch sử.

4. Spark Streaming: Tính toán Windowed Aggregation

Hệ thống sử dụng **Spark Structured Streaming** để xử lý các phân tích phức tạp trên dòng thời gian (Time-series analysis).

4.1 Cơ chế Windowing (Cửa sổ trượt)

Khác với việc tính toán snapshot tức thời, Spark Streaming cho phép nhìn lại một khoảng thời gian ngắn vừa qua để thấy được xu hướng. Hệ thống sử dụng **Sliding Window**:

- **Window Duration (5 phút)**: Tổng hợp dữ liệu trong mỗi khối 5 phút.
- **Slide Duration (1 phút)**: Cứ mỗi 1 phút, một cửa sổ mới sẽ được tính toán lại (gối đầu lên nhau).
- **Watermark (10 phút)**: Cho phép hệ thống chờ đợi dữ liệu bị trễ lên đến 10 phút trước khi chốt kết quả của một cửa sổ và giải phóng bộ nhớ.

4.2 Chi tiết xử lý và Các chỉ số (Metrics)

Dữ liệu từ Kafka được phân tích sâu để tính toán bộ chỉ số KPI đa chiều trong mỗi khung giờ. Toàn bộ logic được tối ưu hóa để chạy trên cụm Spark:

- **Các chỉ số được tính toán:**
 - **avg_speed**: Tốc độ trung bình của tất cả xe trên tuyến trong cửa sổ 5 phút.
 - **msg_count**: Tổng số bản ghi nhận được (thể hiện mật độ dữ liệu).
 - **active_vehicle_count**: Đếm xấp xỉ số lượng xe duy nhất (Distinct Vehicles) xuất hiện.
 - **moving_msg_count**: Số lượng tín hiệu ghi nhận xe đang di chuyển (**speed > 0**).
 - **stopped_msg_count**: Số lượng tín hiệu ghi nhận xe đang dừng (**speed == 0**).
 - **sos_msg_count**: Số lượng tín hiệu báo động khẩn cấp.
 - **ignition_on_msg_count**: Số lượng tín hiệu ghi nhận máy đang nổ.

```
# Logic tính toán chi tiết trong buswaypoint_window.py
route_metrics = (
    parsed_stream.groupBy(
        # Định nghĩa cửa sổ trượt: 5 phút, cập nhật mỗi 1 phút
        window(col("timestamp"), "5 minutes", "1 minute"),
        col("route_no")
    )
    .agg(
        avg("speed").alias("avg_speed"),
        count("*").alias("msg_count"),
        # Sử dụng thuật toán HyperLogLog++ để đếm tập hợp lớn một cách hiệu quả
        approx_count_distinct("vehicle").alias("active_vehicle_count"),
        # Phân loại trạng thái dựa trên điều kiện thực tế
        spark_sum(when(col("speed") > 0,
            1).otherwise(0)).alias("moving_msg_count"),
```

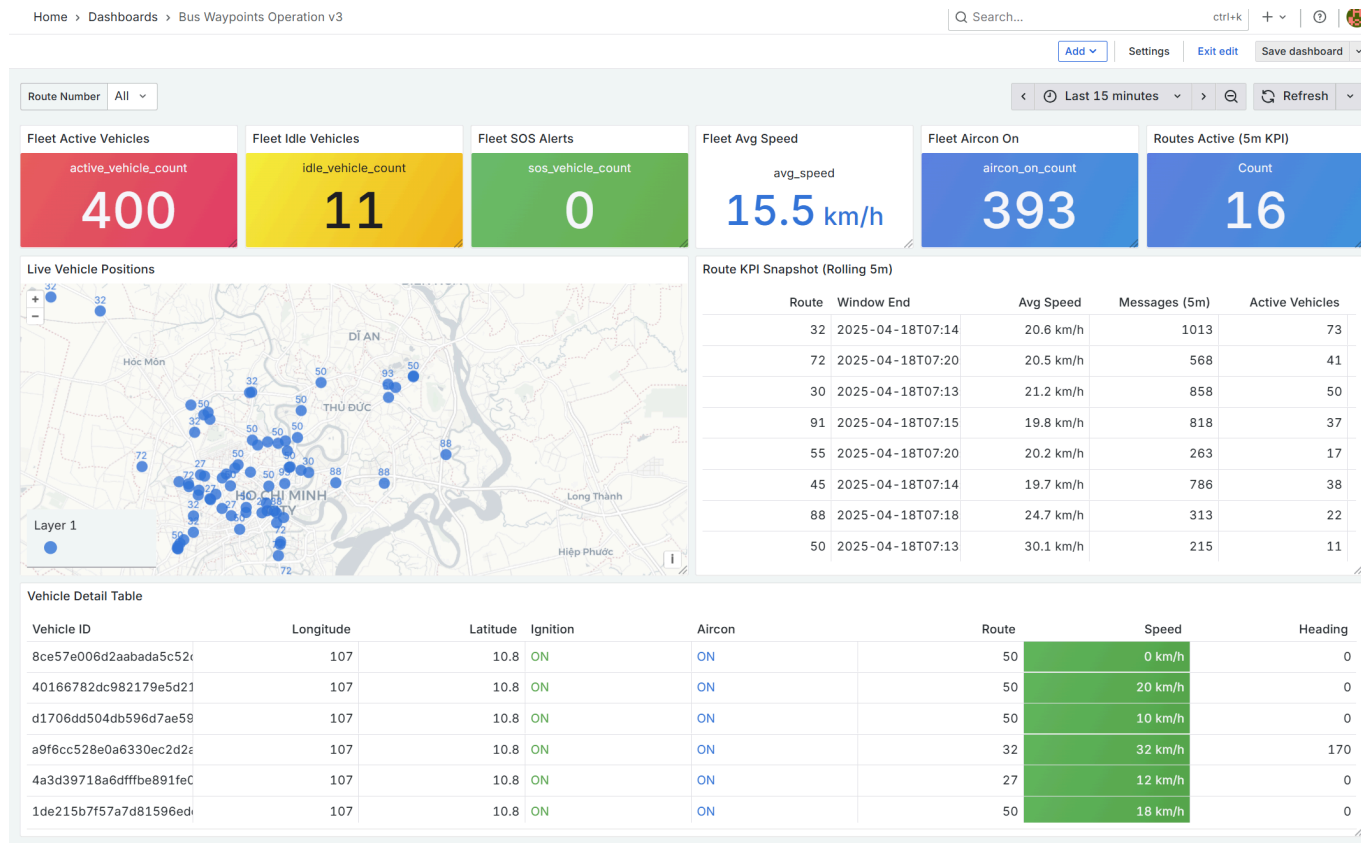
```
spark_sum(when(col("speed") == 0,
1).otherwise(0)).alias("stopped_msg_count"),
spark_sum(when(col("sos") == True,
1).otherwise(0)).alias("sos_msg_count"),
spark_sum(when(col("ignition") == True,
1).otherwise(0)).alias("ignition_on_msg_count")
)
.select(
    "route_no",
    col("window.start").alias("window_start"),
    col("window.end").alias("window_end"),
    "avg_speed",
    "msg_count",
    "active_vehicle_count",
    "moving_msg_count",
    "stopped_msg_count",
    "sos_msg_count",
    "ignition_on_msg_count"
)
.withColumn("updated_at", current_timestamp())
)
```

4.3 Ghi dữ liệu vào Redis

Kết quả sau khi tính toán theo cửa sổ được ghi vào Redis thông qua `foreachBatch`. Điều này cho phép chúng ta vẽ các biểu đồ đường (line chart) thể hiện sự thay đổi về vận tốc hoặc mật độ xe theo từng phút trên Grafana.

5. Ứng dụng thực tế

- **Dashboard Giám sát:** Theo dõi trạng thái vận hành toàn thành phố.



- **Bản đồ trực tuyến:** Hiển thị vị trí xe buýt mà nhờ tốc độ truy xuất cực nhanh.
- **Cảnh báo tức thời:** Tự động thông báo nếu có xe gửi tín hiệu SOS hoặc chạy quá tốc độ trong cửa sổ thời gian gần nhất.
- **Phân tích hiệu quả:** So sánh hiệu suất vận hành giữa các tuyến đường dựa trên dữ liệu trung bình cửa sổ.

6. Độ tươi của dữ liệu (Data Freshness)

Độ tươi của dữ liệu là yếu tố sống còn của Tầng phục vụ. Hệ thống được thiết kế để tối ưu hóa thời gian từ khi sự kiện xảy ra tại xe buýt đến khi hiển thị trên màn hình giám sát:

6.1 Tầng thu thập (Kafka to Redis)

- **Độ trễ xử lý:** < 1 giây.
- **Cơ chế:** Python Consumer sử dụng cơ chế **poll** liên tục với batch size lớn (500 msg) và **pipeline** ghi đồng thời vào Redis.
- **Dấu thời gian (Timestamps):**
 - **event_time:** Thời điểm thực tế xe buýt gửi tín hiệu.
 - **ingested_at:** Thời điểm dữ liệu được nạp vào Redis. Khoảng chênh lệch giữa hai giá trị này giúp đo lường độ trễ (consumer lag).

6.2 Tầng tính toán (Spark Streaming)

- **Độ trễ cửa sổ:** Phụ thuộc vào **Slide Duration** (1 phút). Các chỉ số xu hướng sẽ được cập nhật lại mỗi phút một lần.
- **Trigger:** Spark được cấu hình để kích hoạt tính toán ngay khi có dữ liệu mới trong Micro-batch, đảm bảo kết quả window luôn bám sát thời gian thực nhất có thể.

6.3 Tầng hiển thị (Grafana Dashboard)

- **Tần suất làm mới (Refresh Rate): 5 giây.**
- **Cơ chế:** Dashboard Grafana được cấu hình tự động truy vấn lại Redis mỗi 5 giây. Kết hợp với việc Redis lưu trữ trạng thái "Latest", người điều hành sẽ thấy vị trí xe thay đổi gần như tức thời trên bản đồ.
- **Tối ưu hóa:** Truy vấn từ Grafana tới Redis là các lệnh **HGETALL** hoặc **XRANGE** cực nhẹ, không gây tải cho hệ thống ngay cả khi có hàng chục người cùng theo dõi.